Rent and Shard Splitting

A storage-economics companion to occupancy-driven sharding in F1R3FLY

F1R3FLY Industries — working note

June 10, 2026

A short note. The arithmetic in §5 is an engineering estimate built on public Oracle Cloud list prices as of Q1–Q2 2026; every assumption is stated so the numbers can be re-derived or re-priced.

1 The problem in one paragraph

In the F1R3FLY core—an implementation of the rho calculus [1]—a submitted term is normalized; normalization induces a total order on the syntax tree, which yields content-addressed names and efficient hashes for dispatching COMM events. As a shard serves more deploys, its serviceable hash table fills, and dispatch cost rises toward linear search. Splitting the shard into two relieves that pressure but converts some single-channel locality into cross-shard coordination priced in consensus rounds. *Rent* is the third actor: standing residuals—unconsumed outputs and unmatched for-comprehensions—pay an amortization of real network cost over time, and tenants whose access income falls can opt for cheaper rent deeper in the tree. This note sketches how rent is collected, how it interacts with the split threshold, and what rent actually costs on a concrete ten-node shard.

2 How rent is done

Rent reuses the precharge machinery already present in the reducer; it introduces no new currency.

2.1 End of deploy: the fork

A deploy arrives with a precharge of phloLimit $\times$ phloPrice. Reduction draws it down; what remains is the *residual budget*. The same reduction leaves *residual state*: outputs and for-comprehensions that will sit in RSpace and pay rent. The fork sits exactly where stock Rholang would refund unused phlogiston:

- **Default (no refund flag).** The residual budget is *not* returned. It is sealed as the persistence fund for that deploy's residuals—its new tenants. Unused *compute* budget becomes prepaid *storage*.
- **Refund flagged.** The budget returns to the deployer; the residuals have nothing backing them and are evicted at deploy-end.

The flag is therefore a lifetime declaration—*refund = my residuals are ephemeral, reap them; no refund = they are durable, fund them*—and is the economic twin of the ephemeral/persistent bit in the RSpace 2×2, set by the deployer's choice to reclaim money rather than by a channel annotation.

2.2 Steady state: the draw

Rent debits periodically: a base amortized network cost plus a congestion premium for the tenant's current tier. It is paid against the deployer's account, two claims in priority order—the earmarked residual fund first, then the committed on-chain balance as backstop. The division of labor is clean: *rent* (paid by the owner) covers *standby*; the eventual COMM that consumes a residual is *activation*, paid by the precharge of whoever brings the matching datum. The owner pays to keep an output waiting; the counterparty pays to complete it.

2.3 Depletion: the cascade

When both claims are exhausted: serve notice to the owner; a short grace window; on no top-up, demote the residual one tier down the tree (cheaper rent, via name migration); repeat; eviction at the leaf. Each demotion is a stay of execution bought at lower rent, so depletion is graceful degradation rather than a cliff. It is also forced bid-rent sorting: voluntary sinking is a tenant whose access income fell choosing a longer commute; involuntary sinking is the same motion driven by an empty fund.

Remark 1 (Mortality semantics)**.** *Rent makes residuals* mortal*, which changes the delivery guarantee. In pure rho an output waits forever for its match; here it waits only while solvent. "*COMM*s are eventually delivered" weakens to "*COMM*s are delivered while funded," and an unfunded possible match silently never fires. This is a feature—garbage collection of dead state with an economic reaper—but deployers must see it. It is the mortal-computation story wearing an accountant's hat: the funded residual is soma, the on-chain account is germ line, eviction is the Weismann barrier [7] enforced by an empty wallet.*

Remark 2 (A constraint on migration)**.** *Demotion must be* cheaper than the rent it relieves*, or the cascade costs more than letting the tenant squat. Because names are content-addressed under the normal-form total order, a demotion should be a range handoff (B-tree-style) rather than a rehash. This is a hard bound on how heavy the migration negotiation may be.*

3 Splitting: the density threshold

Let the shard hold N slots, occupancy $\alpha = m/N$, request rate λ, and per-COMM dispatch cost $c(\alpha)$. Splitting roughly halves occupancy per child but converts a fraction χ of traffic into cross-shard coordination at cost X per event (Casper rounds, denominated in phlo), plus a one-time migration cost M amortized over horizon T. Splitting is right when the marginal dispatch saving beats the overhead,

$$\lambda\big[c(\alpha) - c(\alpha/2)\big] \;>\; \chi\lambda X + \tfrac{M}{T}.$$

Write $K = \chi X + M/(\lambda T)$ for the total split overhead in units of probe-cost-per-request. Two regimes:

Conventional open addressing, $c(\alpha) = \frac{1}{1-\alpha}$. The condition $\dfrac{\alpha}{(1-\alpha)(2-\alpha)} = K$ gives, near a full table,

$$\boxed{\alpha^\star \approx 1 - \tfrac{1}{K}}$$

The split density is the reciprocal of the split overhead: overhead of 50 probe-equivalents ⇒ split at 98%; a poorly separable join graph (small K) ⇒ split much earlier.

Elastic hashing (Farach-Colton–Krapivin–Kuszmaul [2]), $c(\alpha) = C \log \frac{1}{1-\alpha}$. The same calculation gives

$$\boxed{\alpha^\star \approx 1 - \tfrac{1}{2}\, e^{-K/C}}$$

The threshold moves *exponentially* closer to full. This is the real engineering content of the elastic-hashing result for F1R3FLY: not faster dispatch per se, but a license to run shards much denser before splitting, because the cost curve has no knee (the bound disproves Yao's conjecture that uniform hashing is optimal [3])—and a threshold far more forgiving to mis-estimating K.

Occupancy is endogenous. α is not a fill fraction but the stationary occupancy of a birth–death process: outputs and receives deposit, COMMs consume. The ephemeral component reaches a queueing equilibrium set by the match rate; persistent receives never die, so occupancy carries a monotone ratchet under the fluctuation. The ratchet guarantees the shard eventually reaches $\alpha^\star$ regardless of throughput—*unless* rent intervenes. With rent, raising the congestion premium lets tenants self-select downward, holding α at an efficient level; the shard splits when the premium collected exceeds K, i.e. when a competitor could profitably stand up a second shard, undercut the premium, and still cover migration and cross-shard joins. Density need not hit a wall; price hits a threshold instead, and both sides are already in phlo.

4 Bid-rent stratification

Rent turns the shard tree into a city. Each resident has access-fee income f_i and sits at the tier maximizing $f_i - r(\text{tier}) - (\text{latency penalty})$—the Alonso–Muth–Mills bid-rent model [4, 5, 6] with tree depth playing the role of distance from the center. Hot contracts bid up rent near the root; cold content cascades toward the leaves. The funnel structure re-emerges as the *equilibrium* of the rent market rather than a design choice.

The elastic-hashing result does its real work at the *cold tail*. Leaf shards serve low λ, so dispatch cost barely matters there, and the FKK bounds permit packing at $\alpha \to 1$ with only logarithmic probe penalty. The equilibrium therefore has an inverse gradient: rent falls and density *rises* down the tree, with cold-storage leaves run nearly full at near-zero rent—which is exactly what makes low rent economically viable (maximum residents per unit of amortized hardware). The ratchet is cured: a persistent receive that stops earning can no longer squat in the hot core; rent forces it to sink, the funnel cascade operating as foreclosure rather than overflow.

5 A concrete rent estimate

Configuration. One shard = ten nodes, each 16 cores, 16 GB memory, 2 TB data, all co-resident in one Oracle Cloud (OCI) region. Aggregate: 160 OCPU, 160 GB RAM, 20 TB block storage.

Unit prices (OCI public list, Q1–Q2 2026 [8]). x86 flexible compute \$0.025 per OCPU-hour; flexible-shape memory \$0.001,5 per GB-hour; Balanced block volume \$0.025,5 per GB-month; the first 10 TB/month of egress is free. One OCPU is one physical core (two x86 vCPUs). We take 730 hours per month.

We use the all-in $C \approx$ \$4,160/month as the quantity rent must amortize. (A three-year Universal Credits commitment would shave 10–35% off; the table below is therefore conservative.)

Table 1. Monthly network cost of the ten-node shard (OCI list prices).

Resource	Quantity	Unit price	Monthly cost
Compute (OCPU-hours)	160 OCPU × 730 h	\$0.025 / OCPU-h	\$2,920.00
Memory (GB-hours)	160 GB × 730 h	\$0.0015 / GB-h	\$175.20
Block storage (Balanced)	20,480 GB	\$0.0255 / GB-mo	\$522.24
Egress ($\leq$ 10 TB/mo)	—	free tier	\$0.00
Subtotal			**\$3,617.44**
Ops / monitoring / LB (15%)			\$542.62
All-in C			**\$4,160.06**

5.1 From deployment rate to tenant population

Rent per tenant is C divided by the standing tenant population N_t. Let each deploy leave on average $\bar{r} = 5$ residuals, accumulating over a 30-day amortization window:

$$N_t \;=\; D \times 30\,\text{days} \times \bar{r}, \qquad r_{\text{base}}(D) \;=\; \frac{C}{N_t}.$$

The binding capacity is *memory* (the hot serviceable table): at 1 kB per residual, 160 GB holds $\approx 1.6 \times 10^8$ hot slots (storage holds $\approx 2 \times 10^{10}$, far larger, so memory binds first).

Table 2. Standing tenants and base rent versus client deployment rate.

Deploys / day D	Standing tenants N_t	Hot-table α	Base rent / tenant-month
100	15,000	< 0.001	\$0.2773
1,000	150,000	0.001	\$0.02773
10,000	1,500,000	0.009	\$0.002773
100,000	15,000,000	0.094	\$0.0002773
1,000,000	150,000,000	0.94	\$0.00002773

The shaded row is the split regime: at 10^6 deploys/day the hot table is $\sim$94% full, exactly where §3 fires. The rent estimate and the split threshold meet at the high-deployment end—rent per tenant is in the tens of microdollars precisely when the shard is about to divide.

5.2 Solvency: rent against access fees

A tenant is *self-sustaining* when its access-fee income covers its rent; otherwise it draws down the precharge fund and, when that empties, sinks. Table 3 gives net rent $= r_{\text{base}}(D) - f$ per tenant-month. Negative entries (shaded) mean the tenant pays its own way and may hold its tier indefinitely; positive entries are a net draw on the residual fund with finite runway.

The diagonal frontier is the bid-rent boundary in action: as the shard densifies (down the table), the access fee needed to stay solvent collapses by an order of magnitude per row. A contract earning a tenth of a cent per month is insolvent on a sparse shard but comfortably self-sustaining once the shard carries a million-plus tenants—which is the cold-tail packing argument of §4 stated in dollars.

Table 3. Net rent per tenant-month, $r_{\text{base}}(D) - f$. Shaded = self-sustaining (surplus).

Deploys/day	Access fee collected f ($/tenant-month)				
	$0	$0.0001	$0.001	$0.01	$0.10
100	+0.2773	+0.2772	+0.2763	+0.2673	+0.1773
1,000	+0.02773	+0.02763	+0.02673	+0.01773	−0.07227
10,000	+0.002773	+0.002673	+0.001773	−0.007227	−0.09723
100,000	+0.0002773	+0.0001773	−0.0007227	−0.009723	−0.09972
1,000,000	+0.00002773	−0.00007227	−0.0009723	−0.009972	−0.09997

5.3 Runway and the cascade

For a net-paying tenant, the residual fund sets the time to first demotion. If a deploy seals, say, $0.01 of leftover phlo behind its five residuals ($0.002/tenant) and collects no access fees, the runway is $0.002/r_{\text{base}}(D)$:

$$D = 100 : \approx 5 \text{ hours}; \qquad D = 10^4 : \approx 22 \text{ days}; \qquad D = 10^5 : \approx 7 \text{ months}.$$

Density makes squatting cheap, which is the point—cold leaves should be hospitable to nearly-idle state, and the same $0.002 buys months of tenancy there but only hours near a contended root.

Sensitivity. Every number scales transparently. Halving the per-residual footprint doubles N_t and halves rent. A Universal Credits discount scales C directly. Raising $\bar{r}$ (residuals per deploy) scales N_t linearly. The congestion premium of §3 adds *on top* of these base figures in the hot core only; in the cold tail it is negligible by construction.

6 One open design knob

The base rent above is charged per occupied slot. The incentive-compatible alternative is to charge for the *probe displacement* a tenant causes at the current α—the true externality, $\propto c'(\alpha)$—which under elastic hashing is small until the shard is nearly full and then rises sharply, automatically pricing tenants out of an over-dense core. The cost is that a tenant's rent then depends on its neighbors' behavior, complicating the name-migration negotiation. Slot rent is simpler and predictable; displacement rent is correct. We lean toward displacement rent in the hot core and flat slot rent in the cold tail, with the crossover at the tier where $c'(\alpha)$ first becomes non-negligible—but we are flagging it as a decision, not settling it here.

Pieces assembled: the rent lifecycle and its mortality semantics (§2), the occupancy-driven split threshold in both hashing regimes (§3), bid-rent stratification (§4), and a concrete dollar estimate on a ten-node OCI shard (§5).

References

[1] L. G. Meredith and M. Radestock. A reflective higher-order calculus. *Electronic Notes in Theoretical Computer Science*, 141(5):49–67, 2005.

[2] M. Farach-Colton, A. Krapivin, and W. Kuszmaul. Optimal bounds for open addressing without reordering. arXiv:2501.02305, 2025.

[3] A. C. Yao. Uniform hashing is optimal. *Journal of the ACM*, 32(3):687–693, 1985.

[4] W. Alonso. *Location and Land Use: Toward a General Theory of Land Rent*. Harvard University Press, Cambridge, MA, 1964.

[5] R. F. Muth. *Cities and Housing: The Spatial Pattern of Urban Residential Land Use*. University of Chicago Press, Chicago, 1969.

[6] E. S. Mills. An aggregative model of resource allocation in a metropolitan area. *American Economic Review*, 57(2):197–210, 1967.

[7] A. Weismann. *The Germ-Plasm: A Theory of Heredity*. Charles Scribner's Sons, New York, 1893.

[8] Oracle Corporation. Oracle Cloud Infrastructure pricing (compute, block volume, and networking price lists). https://www.oracle.com/cloud/price-list/, accessed June 2026.